Formal Verification Workshop

Session 1 — Fundamentals and Real-World Applications

18:00 – 21:00

Built on the COMP63342 Software Security course (University of Manchester)

Tonight

18:00	Welcome + a question for you
18:10	Why software fails
18:35	Exercise 1: spot the bug
18:50	Testing vs. verification
19:15	Break
19:25	From programs to formulas: SAT, SMT
19:50	Bounded model checking
20:15	Exercise 2: predict the verdict
20:30	Who uses this in the real world?
20:50	Quiz + what to install for Session 2

Icebreaker

Name a software failure that made the news.

Any failure. Any decade. Shout it out / drop it in the poll.

When software fails: two classics and a fresh one

Ariane 5, 1996: a 64-bit float squeezed into a 16-bit integer. The conversion overflowed; the rocket self-destructed 40 seconds after lift-off.

> The code was reused from Ariane 4, where that value “could never get that large”.

Therac-25, 1985–87: a race condition between operator keystrokes and the radiation beam controller. Six massive overdoses, several fatal.

> Sequential testing never showed it: the bug needed precise, fast operator timing.

CrowdStrike, 2024: a security update supplied 21 input fields to sensor code expecting 20. The out-of-bounds read blue-screened 8.5 million Windows machines; the bad file was live for just 78 minutes.

> It passed the release tooling: the content validator itself had a bug, and no test ever read the 21st field.

An integer overflow, a race condition, and an out-of-bounds read. Hold that thought — you will hunt all three bug classes yourself in Session 2.

A password check in 11 lines of C

```
char *gets(char *s);  /* removed from C11 — unsafe by design */

int getPassword(void)
{
  char buf[4];
  gets(buf);
  return strcmp(buf, "SMT");
}

/* main: 0 from getPassword => "Access Granted" */
```

session1-demos/getpassword.c — what happens if you type more than three characters?

What gets() does to the stack

The stack frame	
return address	where main continues after the call
saved registers	the caller's state
buf[0] … buf[3]	gets(buf) starts writing here…

gets() never checks the size: byte 5, 6, 7… keep overwriting whatever lies above the buffer — up to the return address.

An attacker chooses those bytes. That is not a crash; that is control of your program.

Could a test suite find this? Only if someone thinks to type the right garbage.

ESBMC finds the overflow itself (CWE-787) — the door. The check is otherwise sound: no input but "SMT" passes it (provable). A non-password input that still gets in is the strncmp logic-flaw demo.

Let's ask a tool instead of an attacker

```
$ esbmc getpassword.c --unwind 8

[Counterexample]
...
Violated property:
  dereference failure: array bounds violated
  CWE: CWE-787 (out-of-bounds write)

VERIFICATION FAILED
```

No input was typed. No exploit was written. The tool proved the overflow is reachable — and printed the input that triggers it.

From counterexample to a runnable test

Ask the same tool the opposite question — "can any input get past the check?" — and it hands back one that does, as a test you can run.

```
  char buf[4];                              /* input ESBMC chooses */
  for (int i = 0; i < 3; i++) buf[i] = nondet_char();
  buf[3] = '\0';
  if (strcmp(buf, "SMT") == 0)
    assert(0);                 /* claim: this is unreachable */

$ esbmc getpassword.c --unwind 8 --generate-ctest-testcase
  → Generated CTest test case: test_case.c

char nondet_char(void) {                  /* test_case.c */
  static const char v[] = { 83, 77, 84 };  /* "SMT" */
  static int i = 0; return v[i++];
}
```

ESBMC found the input that reaches the protected branch. The test compiles, runs, and prints "Access Granted with input SMT" — no one had to guess it.

Docs: esbmc.github.io/docs/c-cpp/ctest-gen

When the check itself is wrong

No overflow, no guessed secret — this time the access check has a one-character bug, and ESBMC walks straight through it.

```
int access_granted(const char *buf) {
  return strncmp(buf, "SMT", strlen(buf)) == 0;
}   /* compares only strlen(buf) chars → any prefix passes */

if (access_granted(buf) && strcmp(buf, "SMT") != 0)
  assert(0);     /* got in WITHOUT the password */

$ esbmc getpassword.c --unwind 8 --generate-ctest-testcase
  buf = { 0, 0, 0 }            → the empty string ""
VERIFICATION FAILED
```

ESBMC grants itself access by typing nothing: "" is a prefix of "SMT", so strncmp compares zero characters and returns 0. The fix is strcmp — compare the whole password.

session1-demos/strncmp-bypass/

Safety and security: two sides of one coin

Safety — the system must not harm the world (Therac-25, Ariane 5).

Security — the world must not harm the system (getPassword).

Same root cause tonight: the program reaches a state it was never meant to reach.

Security goal	Meaning	getPassword breaks it?
Confidentiality	no unauthorised reading	yes — secrets behind the check
Integrity	no unauthorised writing	yes — the stack itself
Availability	service stays up	crash = denial of service

Memory safety: the bug class that won't die

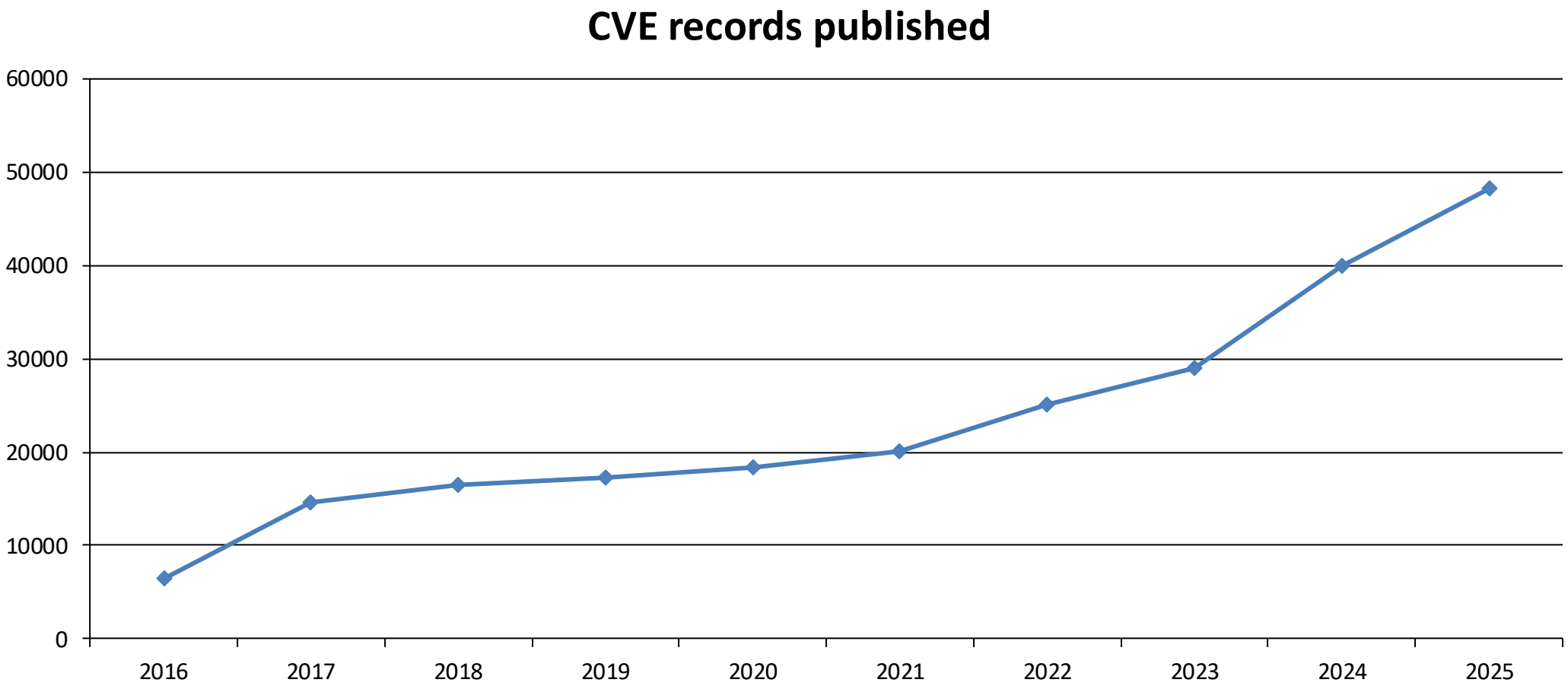
C trusts you completely: every array index, every pointer dereference, every free() is your problem.

Get it wrong and the result is undefined behaviour — the program may crash, corrupt data, or look perfectly fine until it ships.

“Looks fine in testing” is exactly what undefined behaviour does best.

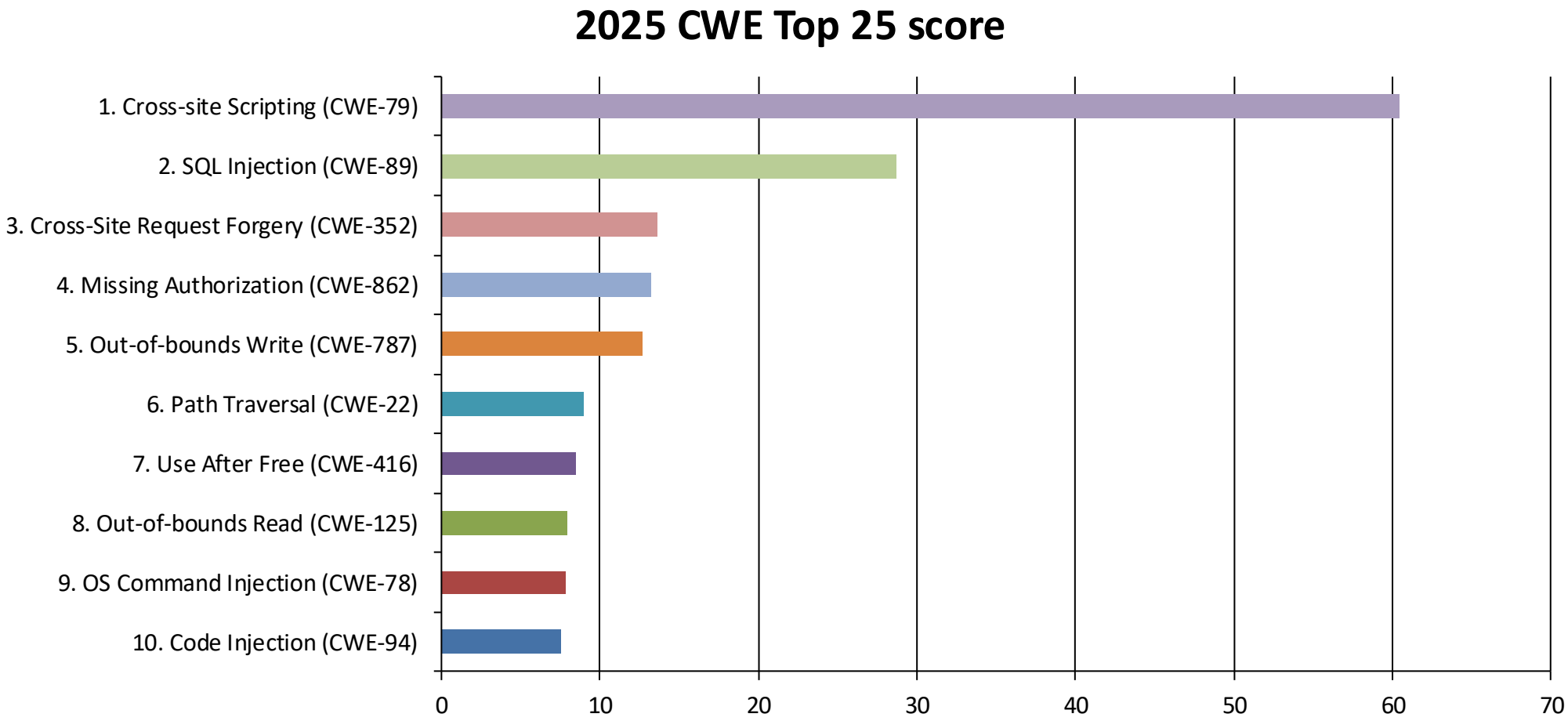
How big is the problem? Let's look at the data.

Vulnerabilities keep climbing



More than 48,000 new CVEs in 2025 — almost tripled since 2020. Sources: nvd.nist.gov · cve.org (2026)

And the same weaknesses top the chart



Memory safety still everywhere: out-of-bounds write #5, use-after-free #7, out-of-bounds read #8 — plus classic buffer overflow at #11. Source: cwe.mitre.org/top25 (Dec 2025)

Industry's conclusion

~70% of the vulnerabilities Microsoft patches every year are memory-safety bugs (MSRC, 2019) — a figure that has barely moved in a decade.

Every one of those products shipped with a serious test suite. Testing did not stop them.

Two industry answers:

- memory-safe languages for new code (Rust, …)
- proving the C/C++ we already run correct — formal verification. Tonight is about this one.

Exercise 1 — Spot the Bug (15 min)

In pairs, 7 minutes, on the handout:

```
int *zPtr;
int *aPtr = NULL;
void *sPtr = NULL;
int number, i;
int z[5] = {1, 2, 3, 4, 5};
sPtr = z;   ++zPtr;   number = zPtr;
number = *zPtr[2];   number = *sPtr;   ++z;
```

For each bug: WHAT is wrong, and what could happen at runtime?

There are at least six. Debrief: one bug per pair.

Verification vs. validation

Validation — are we building the right thing? (ask the users)

Verification — are we building the thing right? (ask the specification)

Tonight is verification: given a program and a property, does every execution satisfy the property?

Why testing cannot prove absence

One int input = 2^{32} cases. At a billion tests per second: about 4 seconds. Fine.

Two int inputs = 2^{64} cases. Same machine: about 585 years.

getPassword's input isn't even bounded — there is no test suite to write.

“Program testing can be used to show the presence of bugs, but never to show their absence.” — E. W. Dijkstra, 1970

A green test suite means: no bug in the cases we tried.

The idea of verification

Turn the program and the property into mathematics, and ask one question:

Can the program reach a state where the property fails?

NO — for all inputs, all paths: a proof.

YES — and here is the exact input: a counterexample.

Both answers are useful. The second one is a free bug report.

Soundness and completeness

Sound — the tool never misses a bug.

- If a sound tool says SAFE, it is safe.

Complete — the tool never raises a false alarm.

- If a complete tool says BUG, it is a real bug.

Testing is complete but not sound (every failure is real; absence proves nothing).

Sound static analysers — e.g. abstract interpretation — are sound but not complete by design (no missed bugs, but false alarms).

Most everyday bug-finders, by contrast, are unsound: they accept missed bugs to scale to real code and keep false alarms manageable.

The techniques as real tools

Technique	In one line	Example tools
Testing & fuzzing	run the code on many inputs	AFL++, libFuzzer, OSS-Fuzz
Symbolic execution	explore paths with a solver; each bug comes with an input	KLEE, SAGE, angr
Bug-finding static analysis	scan for suspicious patterns (unsound — misses bugs)	Coverity, Clang Static Analyzer, CodeQL, Cppcheck
Abstract interpretation	over-approximate every run (sound — but false alarms)	Astrée, Polyspace, Frama-C (Eva)
Bounded model checking	prove properties up to a depth bound	CBMC, ESBMC
Deductive verification	prove code against a spec (needs effort)	Dafny, SPARK (GNATprove), Frama-C (WP)

Tonight's tool, ESBMC, is bounded model checking — and BMC and deductive verifiers both run on the SAT/SMT solvers we meet after the break.

Where does your technique sit?

	Complete (no false alarms)	Incomplete
Sound (no missed bugs)	the dream	abstract interpretation
Unsound	bounded model checking*, testing	code review

* sound only up to its bound. k-induction removes it: prove the inductive step — not just unwind to depth k — and BMC climbs into the Sound row (an unbounded proof when the induction converges; Session 2).

Think–pair–share (5 min): place testing, code review, compiler warnings, and “prove it with maths” on this grid. Defend your placement.

The technique landscape

Technique	What you get	What it costs
Testing	real failures, fast	proves nothing about absence
Static analysis / abstract interp.	sound warnings, scales	false alarms
Symbolic execution	real bugs + the triggering input	path explosion
Bounded model checking	counterexample, or proof up to a bound	the bound
Interactive proof (Coq, Isabelle)	full functional correctness	person-years of effort

Tonight: bounded model checking. At 20:30 we tour who uses the rest.

LLMs find bugs too — but who checks the LLM?

LLMs (Copilot, Claude, Cursor) read code and flag bugs — fast, broad, cheap. They even find real zero-days: Google's Big Sleep found an exploitable buffer underflow in SQLite (2024).

But an LLM is unsound and incomplete: it misses real bugs and hallucinates ones that aren't there — no guarantees. Same corner of the grid as code review.

The win is pairing them — the LLM proposes, a sound checker disposes:

- the LLM suggests bugs, fixes, specs, or inductive invariants;
- the verifier returns a concrete counterexample or a proof — and catches the hallucinations.

ESBMC-AI does exactly this: ESBMC verifies → counterexample + code → LLM proposes a fix → re-verify, loop until proven.

Sources: Google Big Sleep (Project Zero × DeepMind, 2024) · ESBMC-AI — esbmc.github.io/esbmc-ai

The one trick to remember

To prove a property P, ask whether ¬P is satisfiable.

unsat ⇒ no way to violate P exists — a proof

satisfiable, with a model ⇒ a concrete counterexample to P

Every tool tonight, and every lab in Session 2, is this one move in different costumes.

So we need an oracle for “is this formula satisfiable?” — it exists, it is free, and it is absurdly good. After the break.

Break — back at 19:25

SAT: the original hard problem

$(a \vee \neg b) \wedge (b \vee c) \wedge (\neg a \vee \neg c)$

Is there a true/false assignment that makes the whole formula true?

The first problem ever proved NP-complete (Cook, 1971) — in theory, hopeless.

In practice: modern solvers eat formulas with millions of clauses for breakfast. Industrial SAT solving is the closest thing CS has to a superpower.

The two answers a solver can give

sat — and here is a model:

a = true, b = true, c = false ← check it by hand, it works

unsat — no satisfying assignment exists, anywhere.

Not “I didn't find one”. A proof that none exists.

Now recall the trick: encode the bug as the formula.

sat + model = bug found, with the input that triggers it

unsat = that bug is impossible

SMT: SAT that speaks C

In your C program	SMT theory
int, unsigned, wrap-around	bit-vectors — exact machine arithmetic
arrays, indexing	theory of arrays
float, double	IEEE-754 floating point
pointers, malloc/free	memory models built on the above

SMT = SAT + theories. The solver reasons about what your machine actually does — wrap-around, rounding and all — not idealised mathematics.

Worked example: one array access

```
int a[5];           /* valid indices: 0 .. 4 */
a[i] = 0;           /* is this safe? */
```

The property P: $0 \le i \wedge i < 5$

The question to the solver: can the program reach this line with $\neg(0 \le i \wedge i < 5)$?

sat → the model contains the bad i — your counterexample

unsat → the access is safe on every path

Live: a solver in two queries

```
$ python3 z3_demo.py

Query 1: x + y == 42 and x > 100
[y = -59, x = 101]              ← a model, in milliseconds

Query 2: x*x == 2*y*y, x > 0, y > 0 (no overflow)
unsat                            ← a proof
```

Query 2 just proved that √2 is irrational — 32-bit bit-vector edition. unsat is a theorem, not a shrug.

Machine arithmetic is not mathematics

```
INT_MAX     =  2147483647
INT_MAX + 1 =  ?            /* C: undefined behaviour */
                            /* bit-vectors: wraps negative */


int y = x + 1;
int z = y * 2;
assert(z >= 2);    /* surely true for x >= 0 … ? */
```

A solver finds the x that breaks this in milliseconds. In Lab 1 you will encode these three lines yourself and watch it happen.

Solvers as search engines: SEND + MORE = MONEY

```
  S E N D
+ M O R E
---------
M O N E Y
```

A substitution puzzle: replace each letter with a digit. Same letter → same digit; different letters → different digits.

Eight distinct letters — S E N D M O R Y. The words are fixed; what we solve for is the digit hiding behind each letter.

Goal: find the assignment that makes the addition come out true.

Turn the puzzle into arithmetic

A word is just its digits times their place value:

```
SEND  = 1000·S + 100·E + 10·N + D
MORE  = 1000·M + 100·O + 10·R + E
MONEY = 10000·M + 1000·O + 100·N + 10·E + Y
```

So the whole puzzle becomes one equation: SEND + MORE = MONEY.

Plus the rules: every letter is a digit 0–9; all eight are different; S ≠ 0 and M ≠ 0 (no number starts with 0).

Why is MONEY five digits? Two 4-digit numbers can carry into a fifth place — and that carry is at most 1, so M = 1 before we even start.

Let the solver search

Naïvely 10^8 ways to fill in eight letters; “all different, S ≠ 0, M ≠ 0” cuts it to about 1.5 million — still far too many to eyeball.

The solver does not guess blindly: it fixes M = 1, then propagates each column's carry, backtracking when a choice clashes — the way you solve Sudoku.

Z3 returns the unique solution in well under a second:

```
$ python3 send_more_money.py        # in session1-demos/
9567 + 1085 = 10652   (S=9 E=5 N=6 D=7 M=1 O=0 R=8 Y=2)
```

The same engine that solves this finds your bugs: a counterexample is just a satisfying assignment to “break my program”.

Lab 1, Stage 2: you run this yourself.

Bounded model checking: the pipeline

sat → VERIFICATION FAILED + a counterexample trace

unsat → no violation exists within the bound

This is exactly what ESBMC did to getpassword.c before the break — now we open the box.

Step 1 — unwind the loops

```
for (int i = 0; i <= 5; i++)  a[i] = i;
```

--unwind k copies the loop body k times, each copy guarded by the loop condition.

Then one extra check, the unwinding assertion:

> “could the loop have run a (k+1)-th time?”

Bounded — but honest: the tool tells you when its bound was too small, instead of silently missing behaviour.

How to read an --unwind k verdict

The tool prints	It means
VERIFICATION FAILED + trace	a real bug, reachable within k iterations
VERIFICATION SUCCESSFUL	a proof — k covers every possible run of the loop
FAILED: unwinding assertion	the bound was too small — NOT a program bug; raise k and re-run

In Session 2 you will choose k yourself for a loop with at most 50 iterations — keep this table within reach.

“Sound up to its bound”, now made precise.

Step 2 — every assignment gets a fresh name (SSA)

```
int y = x + 1;          →   y₁ = x₀ + 1
int z = y * 2;          →   z₁ = y₁ · 2
assert(z >= 2);         →   check:  ¬(z₁ ≥ 2)  satisfiable?
```

Once every name is assigned exactly once, straight-line code IS a system of equations — no execution needed.

Recognise the three lines? In Lab 1, Stage 3 you translate exactly this function by hand and hand it to Z3. You will be doing ESBMC's job yourself.

Putting it together

$$[\![\text{program}]\!] \;\wedge\; \neg P \;\rightarrow\; \text{SMT solver}$$

sat — the model assigns every input and every choice: replayed in program order, that is the counterexample trace.

unsat — no execution within the bound violates P.

One formula, every path at once. That is why nobody has to enumerate 2^{64} test cases.

Anatomy of a counterexample

```
State 3  file offbyone.c  line 15
  i = 5 (00000000 ... 101)      ← the value the solver chose
...
Violated property:
  array bounds violated: a[i]   ← what broke
  CWE: CWE-787                   ← the vulnerability class
```

Read it bottom-up: first what was violated, then scroll up for the values that caused it.

Those values are a ready-made failing test case — keep them.

Live: the whole pipeline on five lines

```
int main(void)
{
  int a[5];
  for (int i = 0; i <= 5; i++)
    a[i] = i;
}

$ esbmc offbyone.c --unwind 7
```

Predict first: FAILED or SUCCESSFUL? If FAILED — at which i?

Then we read the trace together, bottom-up.

Modelling the environment: a preview

```
int x = nondet_int();                /* every value at once */
__ESBMC_assume(x > 0 && x < 100);    /* now: every value in 1..99 */
assert(property(x));                 /* must hold for ALL of them */
```

One nondeterministic input = all tests of that input at once.

In Session 2 you will use these three lines to expose a bug in a program that verifies green as shipped — the tool only checks what you specify.

Beyond one thread

Two threads of two and one statements already have three interleavings. Real code has billions.

Run the program a million times and the bad interleaving may never show up — Therac-25's bug hid exactly this way.

A model checker enumerates every interleaving up to a bound, including the one that kills your assertion.

Session 2, Lab 4: you write down the killer interleaving before the tool finds it.

Homework for your intuition

```
double w = 0.1 + 0.2;
assert(w == 0.3);
```

FAILED or SUCCESSFUL?

Bring your answer to Session 2 — Lab 4 settles it with a solver that does exact IEEE-754 arithmetic.

Hint: 0.1 in binary is like 1/3 in decimal.

Exercise 2 — Predict the Verdict (15 min)

Groups of 3–4. Three short C programs on the handout (predict1.c, predict2.c, predict3.c).

For each program, write down BEFORE we run anything:

- Will the model checker say VERIFICATION FAILED or SUCCESSFUL?
- If FAILED — which line is the culprit?

Then we run ESBMC live and settle every bet.

Who uses this? — Model checking

Amazon Web Services — CBMC proves memory safety of the TLS library s2n; bounded proofs run in CI on every commit

ESBMC / CBMC — the tools from tonight's demos; compete yearly in SV-COMP on tens of thousands of benchmarks

The counterexample you saw for getpassword.c is the same artefact AWS engineers read when a proof fails

Who uses this? — Symbolic execution & fuzzing

KLEE found 56 serious bugs in coreutils, busybox and Minix — including three in coreutils that had survived 15 years of testing

Microsoft SAGE fuzzed Windows file parsers with symbolic execution; it found ~1/3 of all bugs caught by file fuzzing during Windows 7 development — running last, after every other tool

AFL / OSS-Fuzz — Google fuzzes ~1,000 open-source projects continuously; tens of thousands of bugs found

The pragmatic cousin of verification: no proofs, ruthless effectiveness

Who uses this? — Proofs all the way up

Astrée (abstract interpretation) proved absence of runtime errors in the Airbus A340 fly-by-wire code — 132,000 lines of C — and was then applied to the A380

seL4 — an OS microkernel with a machine-checked proof of functional correctness

CompCert — a C compiler with a proof that compilation preserves semantics; six CPU-years of fuzzing found no wrong-code bugs in its verified core, while every other compiler tested failed

Discussion

If verification can prove the absence of bugs…

why isn't all software verified?

(Let's collect reasons — then I'll tell you which ones the industry actually cites.)

Under the hood — and how to hack on it

Tonight's pipeline was the conceptual one. ESBMC's real pipeline:

> Clang frontend → irep2 (typed IR) → GOTO program → symbolic execution (unwind, SSA) → SMT backends (Z3, Bitwuzla, cvc5)

irep2 is the typed, reference-counted representation everything lowers to and every backend consumes.

Want to add a check, a frontend, or a new node type? Start here:

`github.com/esbmc/esbmc → src/irep2/README.md`

For the curious — and anyone who wants to contribute.

src/irep2/README.md

Closing quiz

1. A tool that never raises a false alarm is called …?
2. ESBMC returns UNSAT for C ∧ ¬P at --unwind 10. What do we know?
3. What's inside a counterexample?
4. Which technique proved the A340 flight-control code free of runtime errors?
5. Why can't testing prove the getPassword program safe?

Before Session 2 — 10 minutes of homework

Follow handouts/setup.md in the repo:

1. `pip install z3-solver`
2. download ESBMC: `github.com/esbmc/esbmc/releases`
3. smoke test: `esbmc labs/lab4/float.c`
   → should print `VERIFICATION FAILED` (yes, failed — that's the point)

Can't make it work? Come at 17:45 — we'll fix it together.